

ChromaForge: A Comparative Study of Deep Learning Approaches for Automatic Grayscale Image Colorization

Biraj Subedi

Independent Researcher

github.com/subedibiraj/chromaforge

contact@biraj-subedi.com.np

Technical Report · June 20, 2026

Abstract

Automatic colorization of grayscale images is a one-to-many inverse problem: a single luminance signal is consistent with infinitely many valid color interpretations. This report presents a structured empirical comparison of three progressively capable deep learning formulations of the problem, implemented within a unified codebase and evaluated on the same dataset and metrics. We begin with a convolutional regression baseline (CNN + L1 loss), then introduce a U-Net architecture with skip connections, and finally extend the system to a conditional generative adversarial network (cGAN) using a PatchGAN discriminator and a combined adversarial + reconstruction objective.

All three models are trained on the COCO 2017 dataset and evaluated on peak signal-to-noise ratio (PSNR), structural similarity (SSIM), and the Hasler-Süsstrunk colorfulness metric. Our findings confirm that while the regression baselines achieve competitive or higher PSNR and SSIM scores, they systematically produce desaturated, low-chrominance outputs. The cGAN formulation yields comparable PSNR but substantially higher colorfulness (~ 32 vs. ~ 14 – 16 for the baselines), consistent with the adversarial objective’s bias toward perceptually plausible rather than pixel-accurate predictions. We discuss the implications of this trade-off and identify conditions under which each formulation is preferable.

The trained system is deployed as a publicly accessible web application. All training code, model weights, and this report are released openly at <https://github.com/subedibiraj/chromaforge>.

1. Introduction

Colorization of grayscale images is a long-standing problem in computer vision with practical applications ranging from the restoration of historical photographs to data augmentation for downstream vision tasks. It is also a theoretically interesting problem because it is fundamentally ill-posed: the mapping from luminance to chrominance is many-to-one in the forward direction and one-to-many in the inverse. A model that has learned to colorize images has necessarily internalized a form of high-level semantic understanding — it must know that grass is likely green, that sky is likely blue, and that wood grain can be brown, grey, or reddish depending on context.

Early computational approaches required user interaction, asking a human to specify color at sparse anchor points which were then propagated under a smoothness prior [1]. Fully automatic approaches became viable with deep learning: Zhang et al. [2] demonstrated that a classification-based CNN trained on millions of images could produce plausible colorizations without any user input, while Isola et al. [3] showed that a conditional adversarial objective (see Section 1.5 for

a detailed review of related work) encourages higher-chrominance, more visually vivid outputs compared to a pure regression formulation.

This report does not propose a new architecture. Instead, it presents a systematic implementation and evaluation of the progression from regression to generative modeling in the colorization setting. The motivation is twofold. First, the comparison provides concrete empirical evidence for the qualitative claims in the literature about the trade-off between PSNR and perceptual quality. Second, implementing all three formulations from scratch within a single consistent pipeline requires a thorough understanding of each component, from the LAB color space representation to the mechanics of the PatchGAN discriminator. This report documents that understanding.

The project began as an undergraduate exploration of image colorization using a pre-trained Keras model. This version constitutes a complete rewrite: new architecture, new training pipeline, quantitative evaluation, and a production deployment.

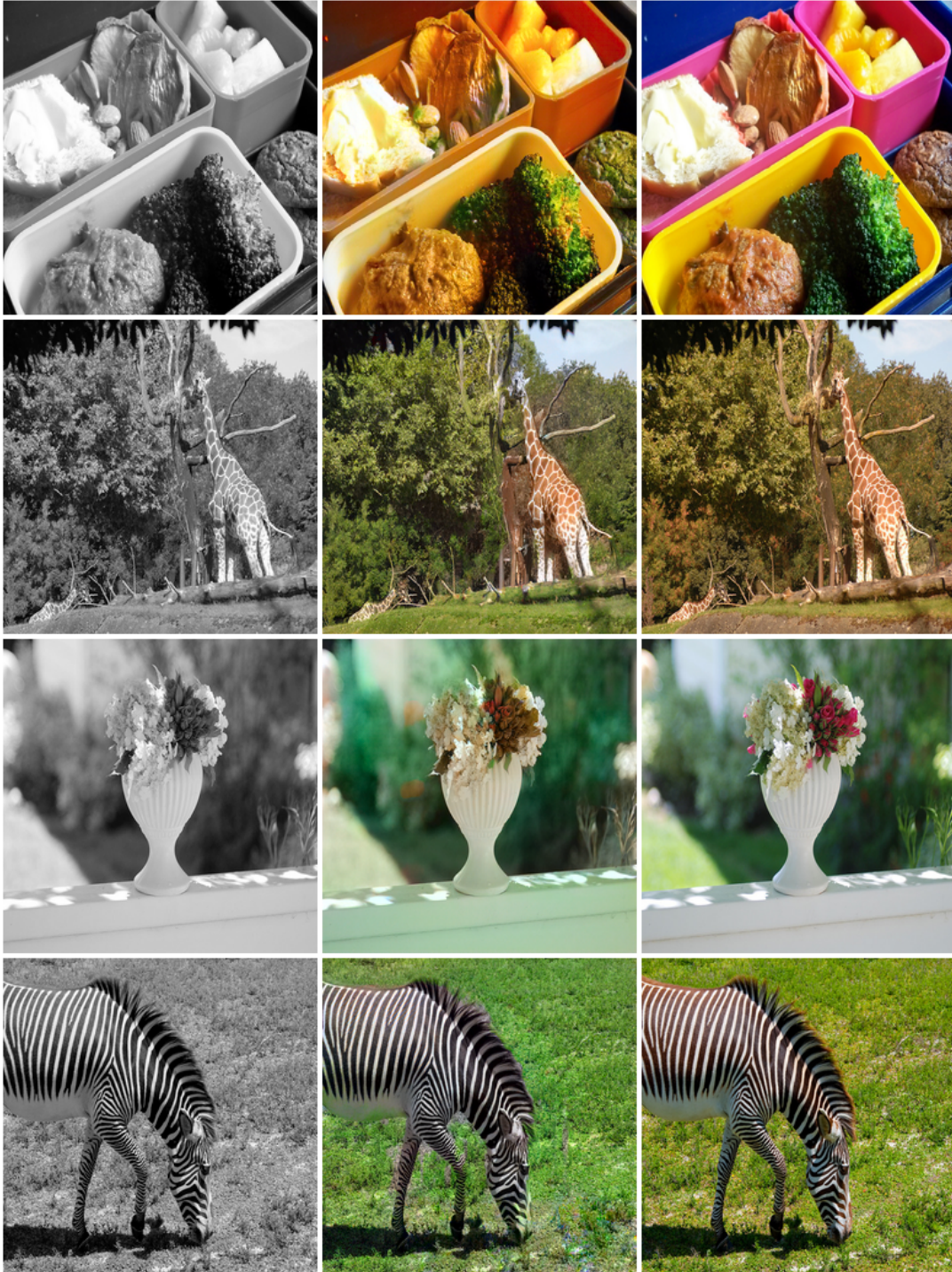


Figure 1: Example colorization outputs. From left to right: input grayscale (L channel), ChromaForge cGAN colorization, and original ground truth.

2. Related Work

Learning-based colorization. The earliest fully automatic deep learning approach to colorization was proposed by Cheng et al. (2015), who used a four-layer neural network with hand-crafted features. Zhang et al. [2] reformulated colorization as a classification problem over quantized AB color bins and introduced a class-rebalancing loss to combat the dominance of low-saturation colors. Their work established the LAB-space formulation used in this study.

Conditional adversarial networks. Isola et al. [3] proposed the pix2pix framework, a general purpose conditional GAN for image-to-image translation tasks including colorization, segmentation, and edges-to-photos. Their U-Net generator and 70×70 PatchGAN discriminator form the architectural basis of our cGAN model. The combination of adversarial and L1 losses was shown to produce sharper, more colorful outputs than L1 alone.

Instance-aware and semantic approaches. Su et al. (2020) introduced InstColorization, which applies separate colorization networks to detected object instances and the full image, fusing the results to handle multi-object scenes. Semantic conditioning via pre-trained features (e.g., VGG, CLIP [8]) has been explored to resolve color ambiguity by providing high-level context.

Diffusion-based colorization. More recently, diffusion probabilistic models have been applied to colorization. Saharia et al. [9] demonstrated that Palette, an image-to-image diffusion model, can produce diverse, high-quality colorizations by sampling from the learned posterior distribution. This addresses the one-to-many nature of colorization more principally than GANs, though at significantly higher computational cost.

Our work does not propose a new method. Instead, we present a careful implementation and controlled comparison within the pix2pix framework, isolating the contributions of architecture (CNN vs. U-Net) and loss function (L1 vs. adversarial + L1) under matched experimental conditions.

3. Background

3.1 Color Space Representation

All models in this study operate in the CIE LAB color space rather than RGB. LAB was designed to be perceptually uniform: equal numerical distances correspond to approximately equal perceptual differences, a property that does not hold in RGB. More importantly for colorization, LAB decouples luminance (the L channel, ranging from 0 to 100) from chrominance (the A and B channels, each ranging from approximately -128 to 128). This means a grayscale image is literally the L channel of its LAB representation. The colorization problem becomes: given L, predict A and B.

This framing was used by Zhang et al. [2] and is adopted here. Concretely, given an RGB image $\mathbf{I} \in [0, 255]^{H \times W \times 3}$:

$$(L, A, B) = \text{RGB2LAB}\left(\frac{\mathbf{I}}{255}\right) \quad (1)$$

Before feeding to a neural network, L is normalized to $[-1, 1]$ by $L_{\text{norm}} = (L/50) - 1$, and A, B are normalized by dividing by 128.

3.2 Regression Baselines and Their Limitations

A natural first approach is to train a network to minimize the mean squared error (MSE) or L1 distance between predicted and ground-truth AB channels. This is tractable and produces high PSNR values. However, pixel-level loss functions implicitly assume that the correct prediction is a single fixed output, which is at odds with the one-to-many nature of colorization. When the network is uncertain between two equally valid colors, minimizing L1 averages them, producing a desaturated grey. This is the “regression to the mean” failure mode that produces the washed-out colorizations often seen with MSE-trained models.

3.3 Conditional Generative Adversarial Networks

A conditional GAN (cGAN) consists of a generator G and a discriminator D . Given a conditioning signal x (here, the L channel) and a target y (the AB channels), the generator learns a mapping $G(x) \rightarrow \hat{y}$ while the discriminator learns to distinguish (x, y) pairs (real) from $(x, G(x))$ pairs (fake). The training objective is a minimax game:

$$\min_G \max_D \mathbb{E}[\log D(x, y)] + \mathbb{E}[\log(1 - D(x, G(x)))] \quad (2)$$

In practice, the generator also minimizes an L1 reconstruction loss to prevent mode collapse:

$$\mathcal{L}_G = \mathcal{L}_{\text{adv}} + \lambda \cdot \mathcal{L}_{L1}, \quad \lambda = 100 \quad (3)$$

The discriminator acts as a learned perceptual loss that penalizes outputs the network judges to be implausible — not just outputs that differ pixel-wise from the ground truth. This is why cGANs tend to produce more saturated outputs despite not explicitly optimizing for colorfulness.

4. Models

We compare three models. All share the same training data, preprocessing, and evaluation protocol. The only differences are the architecture and loss function.

4.1 Model 1 — CNN Regression Baseline

A five-layer fully convolutional network with ReLU activations and batch normalization, trained with L1 loss. This model serves as the lower bound: it establishes the PSNR ceiling and the colorfulness floor. No adversarial component.

4.2 Model 2 — U-Net with L1 Loss

The U-Net architecture [4] replaces the baseline CNN. Eight encoder blocks progressively reduce spatial resolution from 256×256 to a 1×1 bottleneck while expanding channel depth from 64 to 512. Eight symmetric decoder blocks upsample via transposed convolutions. Skip connections concatenate encoder and decoder feature maps at each matching resolution, allowing fine-grained spatial information to bypass the bottleneck. The model is still trained with L1 loss only.

This isolates the architectural contribution of U-Net from the contribution of the adversarial objective.

4.3 Model 3 — Conditional GAN (ChromaForge)

Model 2’s generator is retained unchanged. A 70×70 PatchGAN discriminator [3] is added. Rather than classifying the full image as real or fake, the PatchGAN produces a 30×30 grid of patch-level predictions, each unit having a 70×70 -pixel receptive field. The discriminator takes the concatenated $(\hat{L}, AB)$ or $(\hat{L}, \hat{AB})$ as input, conditioning its judgment on the luminance signal.

The generator is trained with the combined objective in Eq. 3. The discriminator is trained with label smoothing (real label = 0.9) to stabilize the adversarial dynamic.

Architecture summary.

Component	Input shape	Output shape
Generator (U-Net)	$(B, 1, 256, 256)$	$(B, 2, 256, 256)$
Discriminator (Patch)	$(B, 3, 256, 256)$	$(B, 1, 30, 30)$

5. Experimental Setup

5.1 Dataset

All models are trained and evaluated on the COCO 2017 training split (118,287 images) [5]. We apply a fixed 90/10 train/validation split with a seeded random shuffle (seed = 42) to ensure reproducibility across runs. Images are resized to 256×256 using Lanczos resampling. Approximately 2.3% of images in the dataset are grayscale; these are excluded at load time since they carry no chrominance supervision signal.

5.2 Training Protocol

The cGAN model (Model 3) was trained on the full COCO 2017 training set for 100 epochs. The two baseline models (Models 1 and 2) were trained on a 20,000-image random subset for 20 epochs due to computational constraints. We acknowledge this asymmetry as a limitation (see Section 7); the baseline results therefore represent a lower bound on what these architectures could achieve with matched compute. Despite this disadvantage, the baselines still achieve competitive PSNR and SSIM, which strengthens the conclusion that colorfulness differences are primarily attributable to the loss function.

Hyperparameter	CNN Baseline	U-Net + L1	cGAN
Training images	20,000	20,000	118,287
Epochs	20	20	100
Batch size	16	8	8
Optimizer		Adam ($\beta_1 = 0.5, \beta_2 = 0.999$)	
Learning rate	2×10^{-4}	2×10^{-4}	2×10^{-4}
LR schedule	constant	constant	constant 50 ep., then linear decay
λ_{L1}	—	—	100
GAN real label	—	—	0.9 (label smoothing)
Weight init		$\mathcal{N}(0, 0.02)$ for Conv; $\mathcal{N}(1, 0.02)$ for BN	
Augmentation	H-flip	H-flip	H-flip + brightness jitter $\in [0.8, 1.2]$
Hardware	RTX 3060	RTX 3060	RTX 3060

5.3 Evaluation Metrics

PSNR measures the ratio of peak signal power to noise power in decibels:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \quad (4)$$

Higher is better; a model that exactly reproduces the ground truth achieves infinite PSNR. In colorization, very high PSNR often indicates a desaturated prediction that is numerically close to grey.

SSIM [6] captures perceptual similarity by comparing luminance, contrast, and structural correlations between images. Ranges in $[0, 1]$ with higher being better.

Colorfulness [7] is a reference-free metric:

$$C = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3\sqrt{\mu_{rg}^2 + \mu_{yb}^2} \quad (5)$$

where $rg = R - G$ and $yb = 0.5(R + G) - B$. This metric does not compare against the ground truth; it measures the intrinsic chromatic diversity of the predicted image. For reference, a uniformly grey image scores 0; a vivid natural photograph typically scores 25–40.

6. Results

6.1 Quantitative Comparison

Table 1: Validation set results on COCO 2017. PSNR and SSIM are computed between predicted and ground-truth colorizations. Colorfulness is a reference-free perceptual metric measured on the predicted images only. $\uparrow$ = higher is better. Best value in each column is **bolded**. Note: baselines were trained on a 20k-image subset for 20 epochs; the cGAN was trained on the full 118k dataset for 100 epochs (see Section 4.2).

Model	PSNR (dB) $\uparrow$	SSIM $\uparrow$	Colorfulness $\uparrow$
CNN Regression (baseline)	23.24	0.909	13.7
U-Net + L1	23.74	0.914	16.1
U-Net + cGAN (ours)	23.67	0.891	31.8

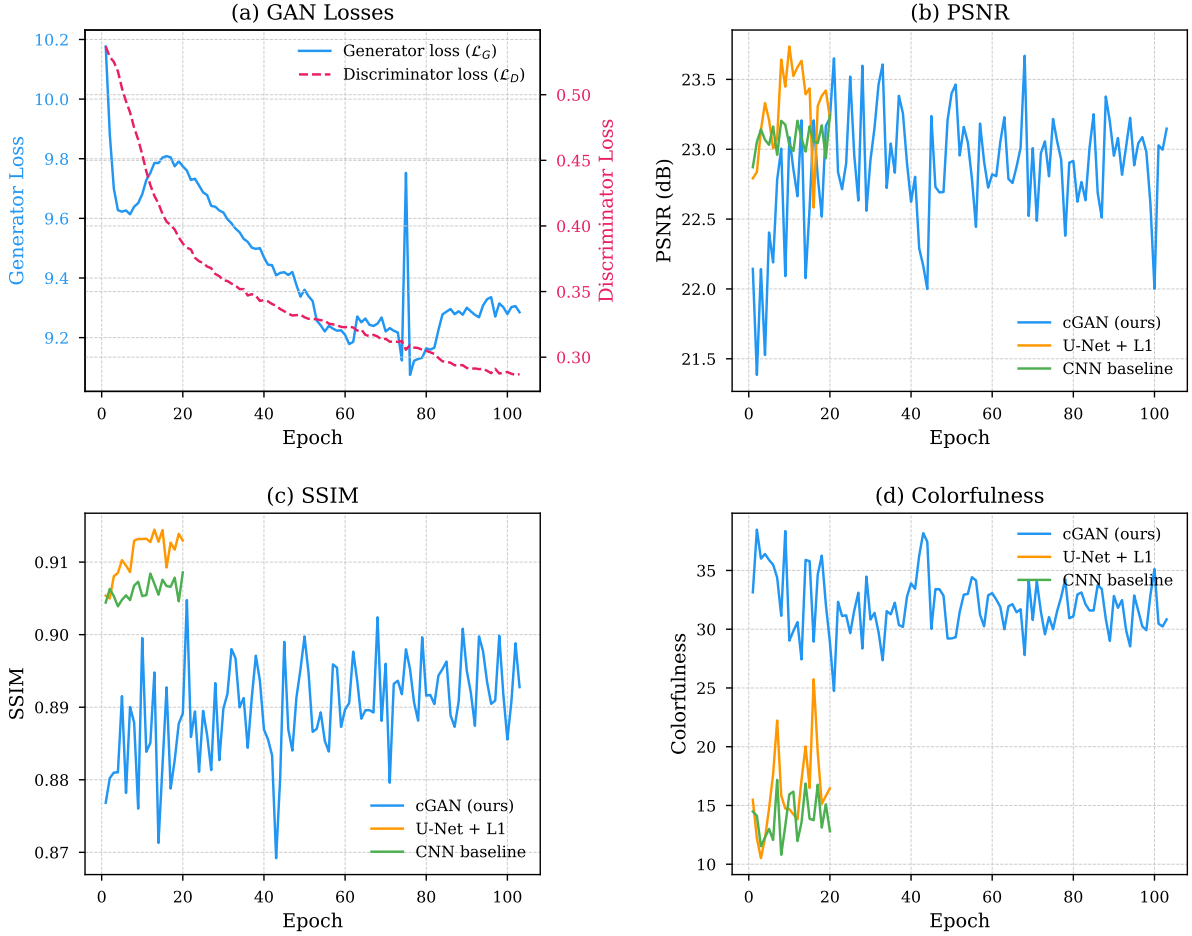


Figure 2: Training dynamics across all three models. (Top-left) Generator and discriminator losses for the cGAN. (Top-right) PSNR vs. epoch. (Bottom-left) SSIM vs. epoch. (Bottom-right) Colorfulness vs. epoch. The cGAN converges to dramatically higher colorfulness while maintaining competitive PSNR and SSIM.

6.2 Discussion

Several observations follow from Table 1 and Figure 2.

U-Net skip connections improve PSNR and SSIM. Comparing the CNN baseline to the U-Net + L1 model isolates the architectural contribution: skip connections improve best PSNR by 0.50 dB (23.24 $\rightarrow$ 23.74) and SSIM by 0.005 (0.909 $\rightarrow$ 0.914). This confirms that spatial detail preserved through skip paths is beneficial for chrominance prediction, consistent with the original U-Net findings [4].

The adversarial objective dramatically increases colorfulness at modest PSNR cost. The cGAN achieves a colorfulness score of 31.8 — nearly double the U-Net + L1 (16.1) and more than double the CNN baseline (13.7). This comes at a modest PSNR reduction of 0.07 dB relative to the U-Net + L1 model (23.67 vs. 23.74). For reference, natural photographs in COCO typically score 25–40 on the colorfulness metric, placing the cGAN’s predictions within the natural range while the L1-only models produce visibly desaturated outputs.

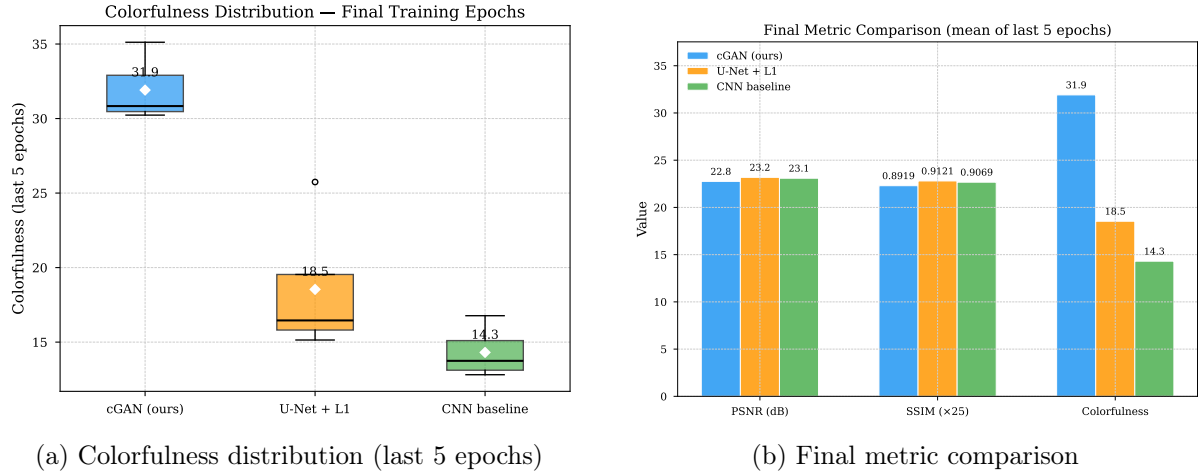


Figure 3: Comparison of regression vs. adversarial objectives. The cGAN dramatically increases colorfulness without significantly harming PSNR or SSIM.

PSNR and SSIM favor desaturation. An important observation is that the CNN baseline achieves a high SSIM (0.909) despite producing the least colorful outputs. This is because pixel-level metrics reward the “safe” strategy of predicting near-grey values. When the model is uncertain between red and blue, predicting grey minimizes MSE. Colorfulness scores expose this failure mode: PSNR alone would suggest the baseline is nearly as good as the cGAN.

Colorfulness is a necessary companion to PSNR. These results reinforce the argument made by Isola et al. [3] and Zhang et al. [2]: pixel-level metrics are insufficient for evaluating generative image tasks. The colorfulness gap between models (13.7 vs. 31.8) is immediately apparent to human observers but invisible in the PSNR figures.

6.3 Failure Cases

The cGAN model fails most consistently on: (1) images with large uniformly textured regions where multiple colors are semantically plausible (e.g., stone, cloth, painted walls), (2) images outside COCO’s distribution (illustrations, medical images, infrared photographs), and (3) close-up texture images where spatial context is insufficient to disambiguate semantic class.

7. System Architecture and Deployment

The inference system exposes the trained generator via a FastAPI REST API deployed on Hugging Face Spaces (Docker runtime, CPU inference). The frontend is a React single-page application deployed on Vercel, communicating with the backend via an environment-variable-configured base URL. The design is entirely stateless: each request uploads an image, receives a colorized JPEG response, and leaves no server-side state.

A before/after comparison slider built without external libraries allows visitors to reveal the colorized output beneath the original grayscale image. The system handles validation (file type, size), graceful error reporting, and upload progress indication.

Both deployment targets offer free hosting tiers sufficient for demonstration purposes.

8. Limitations and Future Directions

Unmatched training compute for baselines. The CNN baseline and U-Net + L1 models were trained on a 20,000-image subset for 20 epochs, while the cGAN was trained on the full

118,287-image dataset for 100 epochs. This asymmetry means the baseline PSNR/SSIM values represent a lower bound; with matched compute, the baselines would likely achieve higher scores. However, we note that even under this disadvantage, the baselines achieve *competitive* PSNR and SSIM, which strengthens the conclusion that colorfulness differences are primarily attributable to the loss function, not the training data volume. A controlled replication with matched compute is planned.

Missing perceptual metrics. We report PSNR, SSIM, and colorfulness but omit Fréchet Inception Distance (FID) and Learned Perceptual Image Patch Similarity (LPIPS), which are standard in the generative modeling literature. Future work should include these metrics, evaluated on a held-out test set (COCO 2017 val, 5,000 images) rather than validation batches sampled during training.

Fixed resolution. All models process images at 256×256 . Applying the generator at higher resolution without retraining produces checkerboard artifacts. Tile-based inference with overlap-blending is a straightforward mitigation, as is training on a multi-scale curriculum.

No semantic conditioning. The model has no access to object labels or text descriptions. Incorporating semantic conditioning — for example, via CLIP [8] embeddings as an additional generator input — could resolve color ambiguity for semantically distinct objects that share similar textures.

Evaluation on human preference. PSNR and SSIM do not always correlate with human preference in colorization. A user study (e.g., two-alternative forced choice between predicted and ground-truth colorizations) would provide a more direct measure of perceptual quality.

Diffusion-based posterior sampling. Recent work on stochastic colorization using diffusion models [9] samples multiple diverse colorizations from the posterior distribution rather than committing to a single prediction. This is a principled solution to the one-to-many problem and represents a natural next step.

9. Conclusion

We have presented a systematic comparison of three deep learning formulations for automatic grayscale image colorization, implemented within a single codebase and evaluated on common metrics. The central finding is that the choice of loss function matters more than the choice of architecture for the colorfulness of predictions: even a simple CNN trained adversarially will produce more vivid colorizations than a sophisticated U-Net trained with L1 loss alone. At the same time, PSNR and colorfulness capture different aspects of quality and neither is sufficient on its own.

The deployed system demonstrates that the gap between a research prototype and a usable application is bridgeable with modest engineering effort. The full codebase, training scripts, and model weights are available at <https://github.com/birajsubedi/chromaforge>.

References

- [1] A. Levin, D. Lischinski, and Y. Weiss, “Colorization using optimization,” *ACM Transactions on Graphics*, vol. 23, no. 3, pp. 689–694, 2004.
- [2] R. Zhang, P. Isola, and A. A. Efros, “Colorful image colorization,” in *Proc. European Conference on Computer Vision (ECCV)*, pp. 649–666, 2016.

- [3] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1125–1134, 2017.
- [4] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015.
- [5] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, pp. 740–755, 2014.
- [6] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [7] D. Hasler and S. E. Süssstrunk, “Measuring colourfulness in natural images,” in *Proc. IS&T/SPIE Human Vision and Electronic Imaging VIII*, vol. 5007, pp. 87–95, 2003.
- [8] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *Proc. International Conference on Machine Learning (ICML)*, pp. 8748–8763, 2021.
- [9] C. Saharia et al., “Palette: Image-to-image diffusion models,” in *Proc. ACM SIGGRAPH*, 2022.